TD2 - Communication on a ring - Solutions

Organization. The exercises that we cannot cover in class are homework: their solutions will be posted, and we may go over some of them later in class. The algorithms of an exercise may be used in the following exercises.

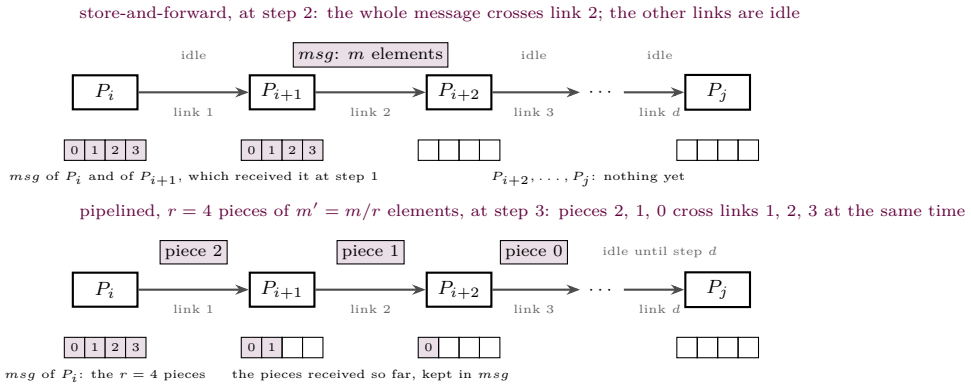
Conventions. The machine is the unidirectional ring of the lecture: p processors $P_0, \dots, P_{p-1}$, each with its own memory; P_k has one outgoing link, to P_{k+1}, and one incoming link, from P_{k-1} (indices modulo p). Every processor runs the same program, with four routines: MyRank() and NumProcs() return the rank k of the processor and p; Send(msg, m) sends to P_{k+1} the m elements of the array starting at msg; Recv(msg, m) receives m elements from P_{k-1} and stores them from msg on. Arrays are passed by name, with their size as a separate parameter (m, n), and p is always NumProcs(); $msg[a \dots]$ as an argument denotes the array starting at its element a (the elements $msg[a], msg[a+1], \dots$), and $M[j]$ the row j of a 2-D array M, an array like any other; an assignment between arrays copies them. Send does not block the processor, Recv blocks it until the message has arrived, and a processor can send and receive at the same time: Send$(\dots) \parallel$ Recv$(\dots)$. A message of m elements costs $L + m\,b$ on a link (L: latency, b: time per element); messages on different links travel at the same time, messages on the same link one after the other; an arithmetic operation costs τ; in practice $\tau \le b \ll L$: transferring an element costs at least as much as operating on it, typically 10 to 100 times more, and the latency is larger by three orders of magnitude again (τ, b: nanoseconds; L: microseconds). The cost of an algorithm is the time until the last processor is done; we distinguish its communication part T_{comm} and its computation part T_{comp}. The following routines of the lecture may be called as building blocks, and so may their pipelined versions of Exercises 2 and 3, which have the same signatures, once they have been established; their costs, with and without pipelining, are the object of Exercises 2 and 3.

Algorithm	Effect
Bcast(k, msg, m)	copies the array msg of P_k into the array msg of every other processor
Reduce$(k, msg, m, \otimes)$	P_k gets in msg the element-wise $\otimes$ of the arrays msg of all the processors; the others keep theirs

Exercise 1

Pipelined point-to-point communication

Processor P_i must send a message of m elements, stored in its array msg, to processor P_j ($j \neq i$); on the ring, the message has to go through $P_{i+1}, \dots, P_{j-1}$ (indices modulo p: the message may wrap around the ring) and crosses $d = (j - i) \bmod p$ links. In the *store-and-forward* protocol, every intermediate processor receives the whole message, then forwards it, which costs $c_{i,j}(m) = d\,(L + m\,b)$: a single link is busy at a time, while the $d - 1$ others are idle. To use them, we *pipeline* the transfer: the message is cut into r pieces of $m' = m/r$ elements (assume that r divides m), sent one after the other; every intermediate processor forwards the piece it received at the previous step while it receives the next one.



a) Write the algorithm SendTo(i, j, msg, m, r), executed by every processor, that performs this pipelined transfer; the processors outside $P_i, \dots, P_j$ do nothing, and P_j receives the message in its own array msg.

b) Find the cost $c_{i,j}(m, r)$ of the pipelined transfer, in terms of r, d, L, m and b.

c) Which value of r minimizes this cost? What is the cost then, and what does it become for large messages? Compare with the store-and-forward cost.

Solution

(a) Number the pieces from 0: piece t is the sub-array $msg[t\,s \ldots (t+1)s-1]$ with $s = m/r$.

P_i sends the r pieces one after the other (line 3). An intermediate processor (line 6: strictly between P_i and P_j along the ring, indices modulo p) receives piece 0 (line 7), then, for $t = 1, \ldots, r-1$, forwards piece $t-1$ while it receives piece t (line 9: the two transfers use its two links at the same time), and finally forwards the last piece (line 10). P_j receives the pieces one after the other (line 5). Every piece lands in the processor's own array msg, at its place: the intermediate processors get a copy of the message too (a temporary buffer would do as well). Forwarding piece $t-1$ while piece t arrives is safe: the two pieces occupy different cells of msg.

SendTo(i, j, msg, m, r)

```
1:  q ← MyRank(); p ← NumProcs(); s ← m/r
2:  if q = i then
3:      for t = 0, ..., r − 1 do Send(msg[t s ...], s)
4:  else if q = j then
5:      for t = 0, ..., r − 1 do Recv(msg[t s ...], s)
6:  else if 0 < (q − i) mod p < (j − i) mod p then
7:      Recv(msg[0 ...], s)
8:      for t = 1, ..., r − 1 do
9:          Send(msg[(t − 1)s ...], s) ‖ Recv(msg[t s ...], s)
10:     Send(msg[(r − 1)s ...], s)
```

(b) Number the links from 1: link l goes from P_{i+l-1} to P_{i+l} (indices modulo p). Piece 0 crosses link 1 at step 1, link 2 at step 2, ..., link d at step d; piece t follows one step behind piece $t-1$ on every link, so it crosses link l at step $l+t$. The last piece, $r-1$, crosses the last link, d, at step $d+r-1$, and every step costs $L + (m/r)\,b$: the pieces on different links travel at the same time. Hence

$$c_{i,j}(m, r) = (d + r - 1)\Big(L + \frac{m}{r}\,b\Big),$$

which gives $d\,(L + m\,b)$ again for $r = 1$. The two diagrams below show, for $d = 3$ links, which piece crosses which link at every step: store-and-forward on the left (the whole message, one link at a time), the pipelined transfer with $r = 4$ pieces on the right; the width of a cell is the duration of the step, $L + m\,b$ on the left and $L + (m/4)\,b$ on the right, here with $L = m\,b/4$.

step	1	2	3
$P_i \to P_{i+1}$	the whole message		
$P_{i+1} \to P_{i+2}$		the whole message	
$P_{i+2} \to P_j$			the whole message

step	1	2	3	4	5	6
$P_i \to P_{i+1}$	0	1	2	3		
$P_{i+1} \to P_{i+2}$		0	1	2	3	
$P_{i+2} \to P_j$			0	1	2	3

(c) Expand the cost as a function of r: $f(r) = c_{i,j}(m, r) = (d-1)\,L + m\,b + r\,L + (d-1)\,\dfrac{m\,b}{r}$; the first two terms do not depend on r. Derivative: $f'(r) = L - (d-1)\,\dfrac{m\,b}{r^2}$, which vanishes when $r^2 = (d-1)\,m\,b/L$:

$$f'(r^\star) = 0 \iff r^\star = \sqrt{\frac{(d-1)\,m\,b}{L}} \qquad \text{(rounded to an integer that divides } m\text{)}.$$

Second derivative: $f''(r) = 2\,(d-1)\,m\,b/r^3 > 0$ for every $r > 0$: f is convex, so $r^\star$ is a minimum, not a maximum (f decreases before $r^\star$ and increases after it). At $r^\star$, the two terms $r\,L$ and $(d-1)\,m\,b/r$ are both equal to $\sqrt{(d-1)\,L\,m\,b}$, hence

$$c_{i,j}(m, r^\star) = (d-1)\,L + m\,b + 2\sqrt{(d-1)\,L\,m\,b}.$$

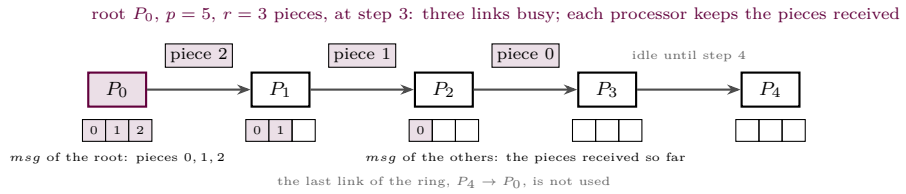
For large messages, $m\,b$ dominates: $c_{i,j}(m, r^\star) \approx m\,b$, the cost of transmitting the message over a *single* link, instead of $d\,m\,b$ with store-and-forward. The pipeline hides the distance: what remains of it is the latency term $(d-1)\,L + 2\sqrt{(d-1)\,L\,m\,b}$, which grows like $\sqrt{m}$ only. For small messages ($m\,b \ll L$), $r^\star < 1$: the best is $r = 1$, store-and-forward, and the d latencies have to be paid anyway.

This pipelined transfer is the idea behind the *cut-through* (or *wormhole*) routing of the network hardware: a switch forwards the units of a message (the *flits*, a few bytes) as soon as they arrive, instead of storing the whole message first, so that a message in transit is spread over all the links of its path. The difference is that, in hardware, a flit only pays a small per-hop delay $t_h \ll L$ and not the latency L of a message, which gives $L + d\,t_h + m\,b$ with nothing to choose; here every piece is a message of its own and pays L on every link, hence the compromise on r.

Exercise 2

Pipelined broadcast on a ring

The Bcast(k, msg, m) of the lecture relays the whole array of the root around the ring, in $(p-1)(L+m\,b)$: again a single link is busy at a time. Pipelining applies: the root cuts its array into r pieces of m/r elements (r divides m) and sends them one after the other; every other processor forwards each piece as soon as it has it, while it receives the next one, and keeps a copy. The number of pieces r is not a parameter: the algorithm chooses it internally, as well as possible (question (c)).



a) Write the pipelined algorithm Bcast(k, msg, m), with the signature of the lecture.

b) Find its cost in terms of p, L, m, b and the number of pieces r.

c) Which value of r minimizes this cost, and is therefore the one the algorithm uses? What is the cost then, and what does it become for large messages? Compare with the $(p-1)(L+m\,b)$ of the lecture.

Solution

(a) The broadcast is a pipelined transfer from the root all the way round the ring, to its left neighbour $P_{(k-1) \bmod p}$: $d = p-1$ links, and every other processor is an intermediate processor of SendTo, which keeps the pieces in its own array msg: exactly what the broadcast wants.

So Bcast(k, msg, m) is simply SendTo$(k, (k-1) \bmod p, msg, m, r)$, with the number of pieces r computed by the algorithm itself (line 2) from the formula of question (c), rounded to a divisor of m and at least 1; this is why SendTo was written with indices modulo p. Unfolded: the root sends the r pieces one after the other; P_{k-1} receives them one after the other; every other processor receives piece 0, forwards piece $t-1$ while it receives piece t, and forwards the last piece. With $r = 1$, this is exactly the Bcast of the lecture.

Bcast(k, msg, m)

```
1: p ← NumProcs()
2: r ← r* = √((p − 2) m b / L)        ▶ see (c)
3: SendTo(k, (k − 1) mod p, msg, m, r)
```

(b) The message crosses $d = p-1$ links, from P_k to P_{k-1}: by Exercise 1(b), the last piece crosses the last link at step $(p-1)+(r-1)$, and

$$T_{comm} = (p + r - 2)\Big(L + \frac{m}{r}\,b\Big).$$

Below, the diagram for $p = 5$, root P_0, $r = 3$ pieces: 6 steps instead of 4, but each one 3 times shorter (up to the latency); from step 3 on, all the links are busy.

step	1	2	3	4	5	6
$P_0 \to P_1$	0	1	2			
$P_1 \to P_2$		0	1	2		
$P_2 \to P_3$			0	1	2	
$P_3 \to P_4$				0	1	2

(c) As in Exercise 1(c) with $d = p-1$: $T_{comm} = (p-2)\,L + m\,b + r\,L + (p-2)\,m\,b/r$ is minimal for

$$r^\star = \sqrt{\frac{(p-2)\,m\,b}{L}}, \qquad T_{comm}(r^\star) = (p-2)\,L + m\,b + 2\sqrt{(p-2)\,L\,m\,b}.$$

This $r^\star$ is the value computed at line 2 of the algorithm (rounded to a divisor of m, at least 1): the algorithm needs only p, m and the machine constants L and b to choose it. For large messages, $T_{comm} \approx m\,b$: the time to push the

message through one link, *independent of* p, whereas the broadcast of the lecture costs $(p-1)(L+m\,b) \approx (p-1)\,m\,b$: the pipelined version is $p-1$ times faster. For small messages, $r=1$ is the best and nothing is gained: the $p-1$ latencies are unavoidable on a ring, the message has to cross $p-1$ links.

Exercise 3

Reduction on a ring

Reduce$(k, msg, m, \otimes)$: every processor holds m elements in its array msg; at the end, the root P_k holds in msg their combination, element by element, with the associative and commutative operation $\otimes$ (sum, max, ...). The lecture gave its principle: a partial result travels around the ring from P_{k+1} to P_k, every processor combining what it receives with its own elements before forwarding it. Combining s received elements, held in a buffer tmp, into s elements of msg is a loop $msg[a+l] \leftarrow tmp[l] \otimes msg[a+l]$, $l = 0, \ldots, s-1$, which costs $s\,\tau$.

a) Write the non-pipelined algorithm Reduce$(k, msg, m, \otimes)$, with the structure of the Bcast of the lecture.

b) Find its communication cost T_{comm} and its computation cost T_{comp}.

c) Pipeline it with r pieces of m/r elements, the number of pieces being chosen internally by the algorithm (question (e)): write the pipelined Reduce$(k, msg, m, \otimes)$, with the same signature. Be careful: a piece can be forwarded only once it has been received *and* combined.

d) Find its costs T_{comm} and T_{comp} (draw what each processor does over time).

e) Which value of r minimizes the total cost? This is the value the algorithm uses.

Solution

(a) The chain starts at P_{k+1}, which has nothing to combine and just sends its array (line 3), and ends at the root (lines 5–6); every other processor receives the partial result in a temporary buffer tmp (line 8), combines it into its own array, element by element (line 9), and forwards its array (line 10).

Write x_i for the array of P_i: after its combination loop, P_q holds $x_{k+1} \otimes \cdots \otimes x_q$, and the root ends with $x_{k+1} \otimes \cdots \otimes x_{k-1} \otimes x_k$, the combination of all the arrays ($\otimes$ is commutative, the order does not matter). Compared with Bcast: the roles of the root and of its neighbours are mirrored (the chain enters the root instead of leaving it), and the combination loop is inserted between the Recv and the Send.

Reduce$(k, msg, m, \otimes)$

```
1:  q ← MyRank(); p ← NumProcs()
2:  if q = (k + 1) mod p then
3:      Send(msg, m)
4:  else if q = k then
5:      Recv(tmp, m)
6:      for l = 0, ..., m − 1 do msg[l] ← tmp[l] ⊗ msg[l]
7:  else
8:      Recv(tmp, m)
9:      for l = 0, ..., m − 1 do msg[l] ← tmp[l] ⊗ msg[l]
10:     Send(msg, m)
```

(b) $p-1$ messages of m elements travel one after the other, one link busy at a time: $T_{comm} = (p-1)(L+m\,b)$. The $p-1$ combination loops, of m operations each, also take place one after the other, since each one waits for the previous message: $T_{comp} = (p-1)\,m\,\tau$, and the total time is $(p-1)(L+m\,b+m\,\tau)$: nothing overlaps.
(c) Pieces of $s = m/r$ elements again; piece t of the partial result is combined into $msg[t\,s \ldots (t+1)s-1]$.

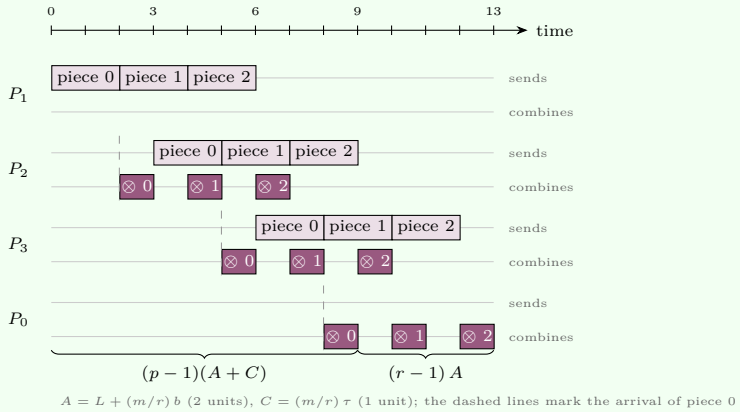
The number of pieces is the $r^\star$ of question (e) (line 2, rounded to a divisor of m). P_{k+1} sends its pieces one after the other (line 4); the root receives and combines them one after the other (lines 6–8). Every other processor receives and combines piece 0 (lines 10–11), then, for $t = 1, \ldots, r-1$, forwards piece $t-1$, combined at the previous iteration, while it receives piece t (line 13), and combines piece t as soon as it is there (line 14); it finally forwards the last piece (line 15). A single buffer tmp is enough: the piece it holds is consumed by the combination loop before the next Recv overwrites it, and the piece being sent lives in msg.

Reduce$(k, msg, m, \otimes)$, pipelined

```
1:  q ← MyRank(); p ← NumProcs()
2:  r ← r⋆ (see (e)); s ← m/r
3:  if q = (k + 1) mod p then
4:      for t = 0, ..., r − 1 do Send(msg[t s ...], s)
5:  else if q = k then
6:      for t = 0, ..., r − 1 do
7:          Recv(tmp, s)
8:          for l = 0, ..., s − 1 do msg[t s + l] ← tmp[l] ⊗ msg[t s + l]
9:  else
10:     Recv(tmp, s)
11:     for l = 0, ..., s − 1 do msg[l] ← tmp[l] ⊗ msg[l]
12:     for t = 1, ..., r − 1 do
13:         Send(msg[(t − 1)s ...], s) ‖ Recv(tmp, s)
14:         for l = 0, ..., s − 1 do msg[t s + l] ← tmp[l] ⊗ msg[t s + l]
15:     Send(msg[(r − 1)s ...], s)
```

(d) Let $A = L + \frac{m}{r}\,b$ be the time of one piece on one link and $C = \frac{m}{r}\,\tau$ the time to combine one piece. The figure

shows what every processor does over time for $p = 4$, root P_0 and $r = 3$: P_1 sends its pieces back to back; on every other processor, piece t is received, combined, then forwarded.



Call h the position of a processor in the chain ($h = 0$ for P_{k+1}, $h = p-2$ for P_{k-1}). Piece t leaves the processor at position h at time $h\,(A+C) + t\,A$: with respect to the previous processor, each piece is delayed by one transfer and one combination, $A + C$; with respect to the previous piece on the same processor, by one transfer, A (the combination of piece t ends exactly when the transfer of piece $t-1$ ends, as the figure shows). The root receives the last piece at time $(p-2)(A+C) + (r-1)\,A + A$ and combines it in C:

$$T = (p-1)(A+C) + (r-1)\,A, \qquad T_{comm} = (p+r-2)\Big(L + \frac{m}{r}\,b\Big), \qquad T_{comp} = (p-1)\,\frac{m}{r}\,\tau.$$

The communication part is that of the pipelined Bcast; the computation part is r times smaller than in (b): the combinations of the different processors now overlap (P_2 combines piece t while P_3 combines piece $t-1$), and only one chain of $p-1$ of them, each r times shorter, is on the critical path. The work has not changed, every processor still combines m elements; it is spread over the pipeline.

(e) Expand the total time of (d) as a function of r:

$$T(r) = (p+r-2)\Big(L + \frac{m}{r}\,b\Big) + (p-1)\,\frac{m}{r}\,\tau = (p-2)\,L + m\,b + r\,L + \frac{K}{r}, \qquad K = (p-2)\,m\,b + (p-1)\,m\,\tau;$$

the first two terms do not depend on r. Derivative: $T'(r) = L - K/r^2$, which vanishes when $r^2 = K/L$:

$$T'(r^\star) = 0 \iff r^\star = \sqrt{\frac{K}{L}} = \sqrt{\frac{m\,\big((p-2)\,b + (p-1)\,\tau\big)}{L}} \approx \sqrt{\frac{(p-2)\,m\,(b+\tau)}{L}}$$

(rounded to an integer that divides m). Second derivative: $T''(r) = 2K/r^3 > 0$ for every $r > 0$: T is convex, so $r^\star$ is a minimum, not a maximum (T decreases before $r^\star$ and increases after it). At $r^\star$, the two terms $r\,L$ and K/r are both equal to $\sqrt{L\,K}$, hence

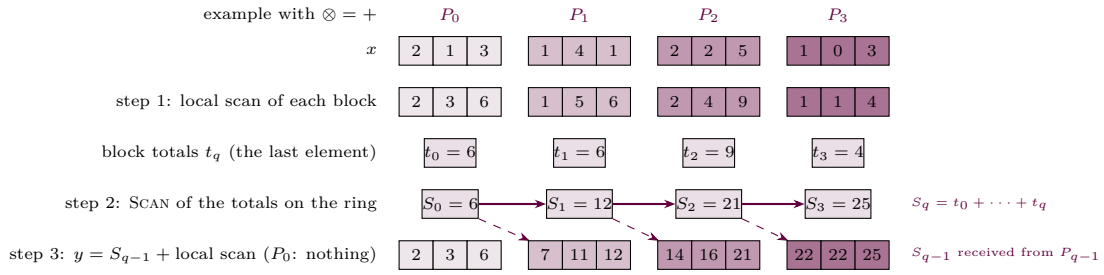
$$T(r^\star) = (p-2)\,L + m\,b + 2\sqrt{L\,K} = (p-2)\,L + m\,b + 2\sqrt{L\,m\,\big((p-2)\,b + (p-1)\,\tau\big)}.$$

Compared with the pipelined Bcast of Exercise 2(c), this is the same formula with b replaced by $b + \tau$ in the square root (up to the factors $p-2$ and $p-1$): the price of the combinations. For large messages, $T \approx m\,b$ again: the computation disappears from the critical path as well, as the term in τ only grows like $\sqrt{m}$. This $r^\star$ is the value the algorithm computes at line 2 (rounded to a divisor of m, at least 1), from p, m and the machine constants L, b, τ.

Exercise 4

Scan on a ring

An array x of N elements is distributed by blocks of $n = N/p$ (p divides N): P_q holds the elements $x[q\,n], \ldots, x[(q+1)n-1]$ in a local array $x[0 \ldots n-1]$. We want its *scan* for an associative operation $\otimes$ (sum, max, ...), as in the PRAM session: $y[i] = x[0] \otimes x[1] \otimes \cdots \otimes x[i]$ for $i = 0, \ldots, N-1$, distributed in the same way: P_q must compute $y[q\,n], \ldots, y[(q+1)n-1]$ in a local array $y[0 \ldots n-1]$. We follow the ScanP of the PRAM session: every processor scans its own block; the p block totals are scanned across the processors; every processor then finishes its block with the result.



a) Write Scan$(v, \otimes)$ on the ring: every processor holds one value v; at the end, P_q holds $v_0 \otimes v_1 \otimes \cdots \otimes v_q$ in v (v_i: the initial value of P_i). Find its costs T_{comm} and T_{comp}. Can it be pipelined like Bcast? Can any algorithm do better on the unidirectional ring?

b) Write ScanP$(x, y, n, \otimes)$, which first scans the local block, then calls Scan on the block totals, and finally finishes the block with a last update. Which quantity does this update combine with the elements of P_q, and how does P_q obtain it? Find the costs T_{comm} and T_{comp} of ScanP.

c) The sequential scan costs $(N-1)\,\tau \approx N\,\tau$. What are the speedup and the efficiency of ScanP? What is the limit of the efficiency for large N, and why is it not 1?

Solution

(a)

The chain of the lecture's Reduce, except that every processor keeps the partial result: P_0 sends v_0 (line 3); P_q receives $v_0 \otimes \cdots \otimes v_{q-1}$ from P_{q-1} in tmp (line 5), combines its own value on the right (line 6: the order matters, $\otimes$ need not be commutative) and, unless it is the last processor, forwards the result to P_{q+1} (line 7). At the end P_q holds $S_q = v_0 \otimes \cdots \otimes v_q$, and it has seen S_{q-1} in tmp on the way.

Scan$(v, \otimes)$

```
1: q ← MyRank(); p ← NumProcs()
2: if q = 0 then
3:     Send(v, 1)
4: else
5:     Recv(tmp, 1)                    ▶ v_0 ⊗ ··· ⊗ v_{q-1}
6:     v ← tmp ⊗ v                     ▶ v_0 ⊗ ··· ⊗ v_q
7:     if q < p − 1 then Send(v, 1)
```

Costs: $p-1$ messages of one element, one after the other, each one waiting for the previous one: $T_{comm} = (p-1)(L+b)$; $p-1$ operations $\otimes$, also one after the other: $T_{comp} = (p-1)\,\tau$; the total time is $(p-1)(L+b+\tau)$, with a single link busy at a time. No pipelining is possible: the messages have a single element, there is nothing to cut into pieces. No algorithm does better: the value of P_0 is needed by P_{p-1}, which is $p-1$ links away on the unidirectional ring, so at least $p-1$ messages have to be sent one after the other, $(p-1)(L+b)$ at least.

(b) Step 1, the local scan (lines 2–3): $y[0] \leftarrow x[0]$, then $y[j] \leftarrow y[j-1] \otimes x[j]$ for $j = 1, \ldots, n-1$, $n-1$ operations. After it, $y[j]$ holds $x[q\,n] \otimes \cdots \otimes x[q\,n+j]$, the combination of the elements of its own block up to position j; the prefix wanted also contains all the elements of the blocks before: $o_q = x[0] \otimes \cdots \otimes x[q\,n-1] = t_0 \otimes t_1 \otimes \cdots \otimes t_{q-1}$, where t_i is the total of the block of P_i (nothing for P_0). This is the quantity the final update combines, on the left, with every element of the block: a single value per processor, to which $P_0, \ldots, P_{q-1}$ contribute, the combination of the block totals before it.

Step 2: the block total t_q is the last element of the local scan (line 4), and Scan on the totals (line 5) gives $t = S_q = t_0 \otimes \cdots \otimes t_q$ on P_q. What P_q needs is $o_q = S_{q-1}$, the value of its *left neighbour*: in the ScanP of TD1 (its line 10), PU j simply read $S'[j-1]$ in the shared memory; on the ring the value has to travel, one more exchange in which every processor sends its S_q to the right (lines 6–7; P_{p-1} has nobody to send to, P_0 nothing to receive). With $\otimes = +$ one could also compute $o_q = S_q - t_q$ locally, but a general $\otimes$ (max, say) has no inverse. Step 3: every element of the block is combined with o_q, on the left (line 8; nothing to do on P_0): $y[j] = o_q \otimes x[q\,n] \otimes \cdots \otimes x[q\,n+j]$, the prefix wanted. The figure of the statement shows the three steps on an example with $\otimes = +$.

ScanP$(x, y, n, \otimes)$

```
1: q ← MyRank(); p ← NumProcs()
2: y[0] ← x[0]
3: for j = 1, ..., n − 1 do y[j] ← y[j − 1] ⊗ x[j]
4: t ← y[n − 1]                        ▶ the block total t_q
5: Scan(t, ⊗)                          ▶ t = S_q = t_0 ⊗ ··· ⊗ t_q
6: if q < p − 1 then Send(t, 1)        ▶ S_q to P_{q+1}
7: if q > 0 then Recv(o, 1)            ▶ o = S_{q-1} = o_q
8: if q > 0 then for j = 0, ..., n − 1 do y[j] ← o ⊗ y[j]
```

Costs: the Scan and one more exchange, on all the links at the same time: $T_{comm} = (p-1)(L+b) + (L+b) = p\,(L+b)$. Computation: $n-1$ operations for the local scan, $p-1$ in the Scan, n for the final update:

$T_{comp} = (2n + p - 2)\,\tau \approx (2n + p)\,\tau$. Remark: P_q had already received S_{q-1} inside SCAN (the *tmp* of its line 5); a SCAN that also returns it would save the last exchange, $T_{comm} = (p-1)(L+b)$; we keep the version that calls SCAN as it is.
(c) $T = T_{comp} + T_{comm} \approx (2n + p)\,\tau + p\,(L + b)$ with $n = N/p$, and $T_{seq} = (N-1)\,\tau \approx N\,\tau$:

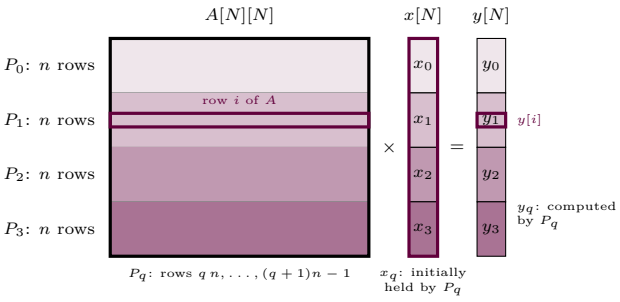
$$S = \frac{T_{seq}}{T} \approx \frac{N}{2N/p + p + p\,(L+b)/\tau}, \qquad E = \frac{S}{p} \approx \frac{1}{2 + \dfrac{p^2}{N} + \dfrac{p^2(L+b)}{N\,\tau}}.$$

When N grows ($N \gg p^2(L+b)/\tau$), the two last terms vanish and $E \to 1/2$: the speedup tends to $p/2$, not to p. The communication is not the reason, it is amortized; the reason is the work: every processor goes over its block twice, once to scan it and once to update it, $2n$ operations instead of n, so the p processors perform $2N$ operations in total against $N-1$ sequentially. The two passes are unavoidable here, since the offset of a block is known only once the totals have been scanned (computing the total first, then scanning once with the offset as the initial value, also takes $2n$ operations).

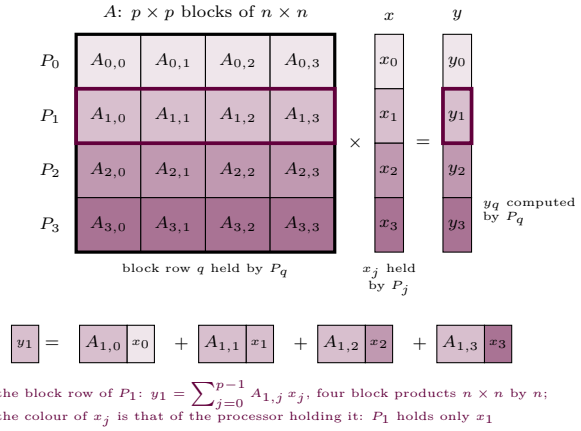
Exercise 5

Matrix–vector product on a ring

Let A be an $N \times N$ matrix and x a vector of N elements; we want to compute $y = A\,x$, that is $y[i] = \sum_{j=0}^{N-1} A[i][j]\,x[j]$. The data are distributed by blocks of $n = N/p$ (p divides N): P_q holds the rows $q\,n, \ldots, (q+1)n-1$ of A in a local array $A[0\ldots n-1][0\ldots N-1]$, and the elements $x[q\,n], \ldots, x[(q+1)n-1]$ of x in a local array $x[0\ldots n-1]$; it must compute the elements $y[q\,n], \ldots, y[(q+1)n-1]$ of y in a local array $y[0\ldots n-1]$.



Seen by blocks: cut A into $p \times p$ blocks of $n \times n$ elements, $A_{q,j}$ being the block at the rows of P_q and the columns $j\,n, \ldots, (j+1)n-1$, and cut x and y into p blocks of n elements, x_j and y_j. Then $y_q = \sum_{j=0}^{p-1} A_{q,j}\,x_j$: the block y_q of P_q is the sum of p small matrix–vector products, one per block of x, each one an $n \times n$ block times n elements. P_q holds the whole block row $A_{q,0}, \ldots, A_{q,p-1}$ of A, but a single block of x, its own x_q.



a) Which elements of x does P_q need to compute its block of y? Which processors hold them? How many phases of computation and of communication must take place, and what should be computed and communicated in each phase?

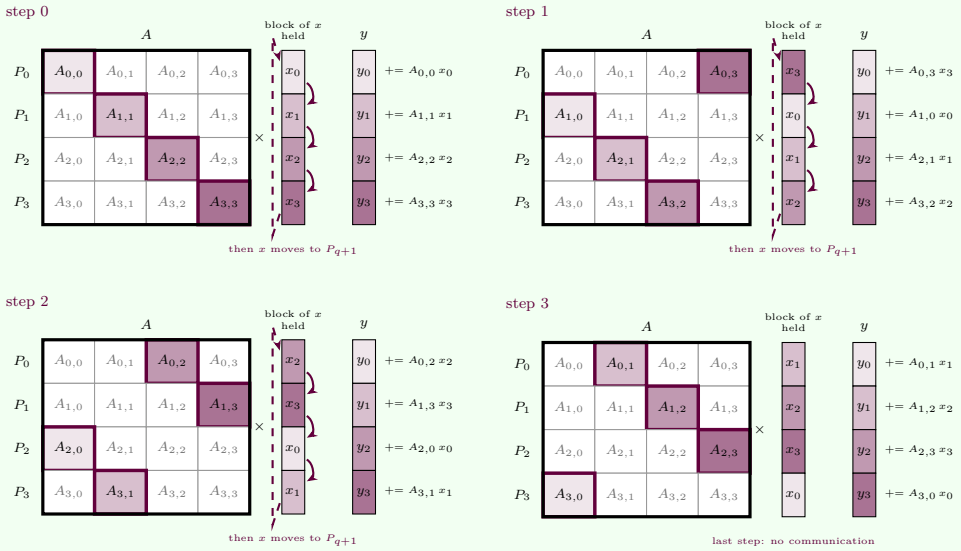
b) Design MatVec(A, x, y, n) on the ring, where A is the local array of n rows and $N = n\,p$ columns of the processor, and x and y its local arrays of n elements: the blocks of x travel around the ring; at every step, every processor uses the block it holds, then passes it to its right neighbour.

c) Find its communication cost and its computation cost.

d) The sequential product costs $2N^2\,\tau$. What are the speedup and the efficiency of MatVec? When is the efficiency close to 1?

Solution

(a) $y[i] = \sum_{j=0}^{N-1} A[i][j]\,x[j]$ uses the *whole* vector x, for every row i: P_q needs all the p blocks of x, and it holds only its own; the block $x_j = x[j\,n \dots (j+1)n-1]$ is held by P_j. In terms of blocks: $y_q = \sum_{j=0}^{p-1} A_{q,j}\,x_j$, where $A_{q,j}$ is the block of A at the rows of P_q and the columns $j\,n, \dots, (j+1)n-1$ (the local columns $j\,n+l$, $l = 0, \dots, n-1$). Hence p phases of computation, one per block of x: in each phase, P_q computes one block product $A_{q,j}\,x_j$ with the block x_j it holds at that moment and adds it to y_q (n^2 multiplications and n^2 additions). Between two computation phases, a communication phase makes every block of x move to the next processor: P_q sends the block it has just used to P_{q+1} and receives a new one from P_{q-1}; $p-1$ communication phases, after which every block of x has visited every processor. The figure shows the $p = 4$ steps: at step s, P_q holds x_j with $j = (q-s) \bmod p$ (its own block at step 0, its left neighbour's at step 1, ...), uses the block $A_{q,j}$ of its rows (filled in the colour of x_j: the diagonal blocks at step 0, then the sub-diagonals, wrapping around) and updates y_q; then the blocks of x move one processor to the right (arrows).



(b) Make the blocks of x travel: at step s ($s = 0, \dots, p-1$), P_q holds the block $j = (q-s) \bmod p$ of x: its own at step 0, the block of its left neighbour at step 1, and so on.

It multiplies the block $A_{q,j}$ by it and accumulates the result into y (lines 6–8), then passes the block to its right neighbour while it receives the next one from its left neighbour (lines 9–11): all the blocks of x rotate around the ring together, one position per step, and after $p-1$ rotations every processor has used every block, as the figure of (a) shows. The two buffers are swapped before the exchange (line 10), so that the block being sent is not overwritten by the one arriving; x itself is never modified.

MatVec(A, x, y, n)

```
1:  q ← MyRank(); p ← NumProcs()
2:  for i = 0, ..., n − 1 do y[i] ← 0
3:  tempR[0 ... n − 1] ← x[0 ... n − 1]
4:  for s = 0, ..., p − 1 do
5:      j ← (q − s) mod p                    ▶ block of x held
6:      for i = 0, ..., n − 1 do
7:          for l = 0, ..., n − 1 do
8:              y[i] ← y[i] + A[i][j n + l] · tempR[l]
9:      if s < p − 1 then
10:         tempS ↔ tempR
11:         Send(tempS, n) ‖ Recv(tempR, n)
```

(c) Communication: $p-1$ exchanges of a block of n elements, all the links busy at every exchange: $T_{comm} = (p-1)(L + n\,b) = (p-1)\big(L + \frac{N}{p}\,b\big)$. Computation: at every step, n rows times n elements, n^2 multiplications and n^2 additions; p steps: $T_{comp} = 2\,p\,n^2\,\tau = \frac{2N^2}{p}\,\tau$, the sequential work divided by p.

(d) $T = T_{comp} + T_{comm} = \frac{2N^2}{p}\,\tau + (p-1)\left(L + \frac{N}{p}\,b\right)$ and $T_{seq} = 2N^2\,\tau$:

$$S = \frac{T_{seq}}{T} = \frac{2N^2\,\tau}{\frac{2N^2}{p}\,\tau + (p-1)\,L + (p-1)\,\frac{N}{p}\,b};$$

dividing the numerator and the denominator by the computation time $\frac{2N^2}{p}\,\tau$:

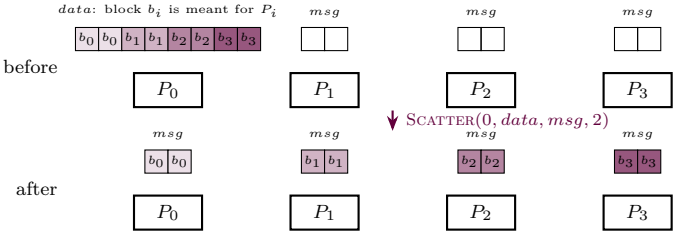
$$S = \frac{p}{1 + \frac{p\,(p-1)\,L}{2N^2\,\tau} + \frac{(p-1)\,b}{2N\,\tau}}, \qquad E = \frac{S}{p} = \frac{1}{1 + \frac{p\,(p-1)\,L}{2N^2\,\tau} + \frac{(p-1)\,b}{2N\,\tau}}.$$

The computation grows like N^2 and the communication like N: the efficiency tends to 1 when N grows, more precisely when $N^2 \gg p^2 L/\tau$ (the latencies are amortized) and $N \gg p\,b/\tau$ (the transfers are amortized). With $L \approx 10^4\,\tau$ and $b \approx 10\,\tau$, this means $N \gg 100\,p$: a few hundred rows per processor. For a fixed N, the efficiency drops when p grows: with p processors, each one holds N/p rows and the $p-1$ latencies are paid in full.

Exercise 6

Scatter on a ring

In many applications, the data are initially held by a single processor (the one that read them from a file, say) and must be distributed in equal parts to all the processors, instead of starting from an already distributed configuration. Scatter$(k, data, msg, m)$ does exactly this: the root P_k holds an array $data$ of $p\,m$ elements, made of p consecutive blocks of m elements, the block $b_i = data[i\,m \ldots (i+1)m - 1]$ being meant for P_i; at the end, every P_i holds its block b_i in its array msg of m elements.

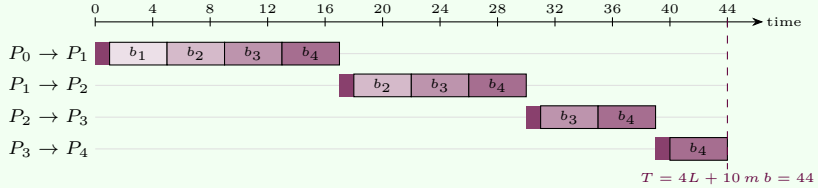


a) Design Scatter$(k, data, msg, m)$ on the ring. Start with the naive version: the root keeps its block and sends all the others to its right neighbour, which keeps its block and sends the rest on, and so on. Then make the whole operation a pipeline: in which order should the root send the blocks, so that no block waits behind another one? Finally, write a third version made of $p-1$ calls to the SendTo of Exercise 1.

b) Find the communication costs of the three versions. Explain why the cost of the pipelined version is the same as that of Bcast although $p-1$ different blocks are sent, and when the version based on SendTo is as good.

Solution

(a) The naive version: the root keeps its own block and sends the $p-1$ others, in ring order, to its right neighbour; P_{k+1} receives $p-1$ blocks, keeps the first one, its own, and sends the $p-2$ others on; P_{k+2} receives $p-2$ blocks, keeps the first one and forwards $p-3$; and so on until P_{k-1}, which receives its block alone. The diagrams of this exercise show what every link carries over time, for $p = 5$, root P_0, $L = 1$ unit and $m\,b = 4$ units; the dark head of every message is its latency. Here: one link at a time, and the blocks are transmitted again on every link they cross: b_4 crosses the four links one after the other.

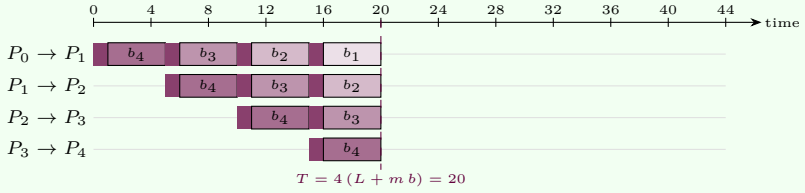


The processor at distance $d = (q - k) \bmod p$ from the root receives $p - d$ blocks, its own first (line 10), keeps it (line 11) and forwards the $p - d - 1$ others (line 12); the blocks of $P_{k+1}, \dots, P_{k-1}$ are contiguous in *data* when $k = 0$, otherwise the root first copies them in that order into *tmp* (lines 5–7). Simple, but a single link is busy at a time: each processor waits for the whole message before sending the rest on.

SCATTER$(k, data, msg, m)$, NAIVE

```
1:  q ← MyRank(); p ← NumProcs()
2:  d ← (q − k) mod p                                  ▶ distance to the root
3:  if q = k then
4:    msg[0 . . . m − 1] ← data[k m . . . ]            ▶ its own block
5:    for i = 1, . . . , p − 1 do
6:      j ← (k + i) mod p
7:      tmp[(i − 1) m . . . i m − 1] ← data[j m . . . ]
8:    Send(tmp, (p − 1) m)                             ▶ the others, in ring order
9:  else
10:   Recv(tmp, (p − d) m)                             ▶ its own block first
11:   msg[0 . . . m − 1] ← tmp[0 . . . m − 1]
12:   if d < p − 1 then Send(tmp[m . . . ], (p − d − 1) m)
```

Better: a pipeline, as in the pipelined SendTo or the pipelined Bcast with the blocks as the pieces: the root sends one block per step without waiting, every block is relayed one link per step, and all the links work at the same time. Two differences with the broadcast. First, the root must start from the *other end* of its array: the farthest block first, in the *reverse order of the distances*: first the block b_{k-1} of P_{k-1}, the farthest processor ($p - 1$ links away), then b_{k-2}, ..., and last b_{k+1} for its neighbour; at step t it sends the block of $P_{(k-t) \bmod p}$. Second, a processor does not wait for all the pieces, since it needs only one of them: it forwards the blocks that are not for it and stops at its own. The diagram: from its first use on, every link carries a block at every step, and all the blocks arrive at step $p - 1$.

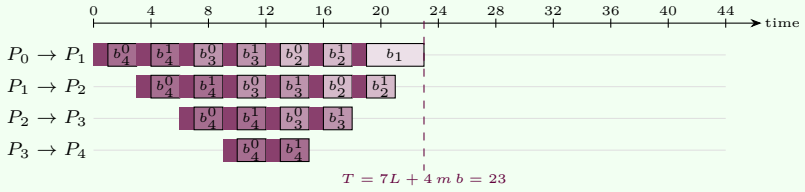


A block then never waits: when b_{k-t} reaches a processor, the block b_{k-t+1} sent before it has already left. Processor P_q, at distance $d = (q - k) \bmod p$ from the root, receives its first block at step d (line 7), forwards every block it receives except the last one (lines 8–10: $(k - 1 - q) \bmod p = p - d - 1$ of them), and keeps the last one, which is its own (line 11). The two buffers are swapped before every step, so that the block being forwarded is not overwritten by the one arriving. The root copies its own block locally (line 5).

SCATTER$(k, data, msg, m)$

```
1:  q ← MyRank(); p ← NumProcs()
2:  if q = k then
3:    for t = 1, . . . , p − 1 do
4:      Send(data[((k − t) mod p) m . . . ], m)
5:    msg[0 . . . m − 1] ← data[k m . . . k m + m − 1]
6:  else
7:    Recv(tempR, m)
8:    for t = 1, . . . , (k − 1 − q) mod p do
9:      tempS ↔ tempR
10:     Send(tempS, m) ‖ Recv(tempR, m)
11:   msg[0 . . . m − 1] ← tempR[0 . . . m − 1]
```

A third version reuses the SendTo of Exercise 1 as a building block: $p - 1$ calls, one per destination, farthest first (so that an intermediate processor, whose *msg* is overwritten by every block it relays, ends with its own block), each call pipelined with the number of pieces of 1(c) for its distance. The calls do not wait for each other: Send does not block, so the root issues the messages of all its calls back to back and they queue on its link, while every other processor, which only receives and forwards, consumes them in the order they come; the transfers overlap, and the ring behaves as a single pipeline, exactly as in the previous version. The diagram, with $r = 2$ pieces for the three farthest transfers and $r = 1$ for the neighbour: the root's link carries the pieces of the four calls back to back, and the three extra messages cost three latencies more than the pipelined version.



Every call is collective: all the processors execute it, in the role SendTo gives them (source, intermediate or destination); the number of pieces is the $r^\star$ of 1(c) for the distance of the call (line 4), and the root passes the block where it is in *data*, the others their *msg* (lines 5–6). With $r = 1$ (no pieces), the calls reproduce the pipelined Scatter message for message: the root's sends are the same, and the receive-and-forward sequence of the other processors is what the calls unfold to. The pieces of 1(c) are useless here, the pipeline being full from the start: they only add latencies, see (b).

Scatter$(k, data, msg, m)$, with SendTo

```
1: q ← MyRank(); p ← NumProcs()
2: for t = 1, ..., p − 1 do
3:     j ← (k − t) mod p; d ← p − t                    ▶ farthest first
4:     r ← sqrt((d − 1) m b/L)                          ▶ the r* of 1(c)
5:     if q = k then SendTo(k, j, data[j m ...], m, r)
6:     else SendTo(k, j, msg, m, r)
7: if q = k then msg[0 ... m − 1] ← data[k m ...]
```

(b) Naive version: step t ($t = 1, \dots, p-1$) is the message of $p - t$ blocks on the link $P_{k+t-1} \to P_{k+t}$, which costs $L + (p-t)\,m\,b$, and the steps take place one after the other:

$$T_{naive} = \sum_{t=1}^{p-1} \big(L + (p-t)\,m\,b\big) = (p-1)\,L + \frac{p\,(p-1)}{2}\,m\,b.$$

The first message already pushes the $p-1$ blocks through the root's link, in $(p-1)\,m\,b$; but the same blocks are then transmitted again on the following links, $p-2$ of them on the second, $p-3$ on the third, ..., one link at a time: $p\,(p-1)/2$ block transmissions one after the other, instead of $p-1$.
Pipelined version: the block b_{k-t} leaves the root at step t and crosses one link per step: it crosses link l (from P_{k+l-1} to P_{k+l}) at step $t + l - 1$ and reaches $P_{k-t} = P_{k+p-t}$ after $p - t$ links, at step $t + (p-t) - 1 = p - 1$. All the blocks arrive at step $p-1$, whatever their distance, and $T_{comm} = (p-1)(L + m\,b)$: the same latency term as the naive version, a transmission term $p/2$ times smaller. The diagram shows the blocks on the links for $p = 5$ and root P_0: from its first use on, every link carries a block at every step.

step	1	2	3	4
$P_0 \to P_1$	b_4	b_3	b_2	b_1
$P_1 \to P_2$		b_4	b_3	b_2
$P_2 \to P_3$			b_4	b_3
$P_3 \to P_4$				b_4

Why the same cost as the non-pipelined Bcast, although $p-1$ different blocks are sent? The $p-1$ blocks all leave the root through its single outgoing link, one per step: the root alone needs $p-1$ steps of $L + m\,b$, whatever the algorithm. Bcast sends a single block but relays it one link at a time: $p-1$ steps as well, with one link busy at each step. Scatter keeps all the links busy at the same time: while the root sends its second block, the first one is already crossing the next link, and so on; the reverse order is what makes it a pipeline, the blocks follow each other one step apart on every link, exactly like the pieces of Exercise 2, and the last block, for P_{k+1}, arrives one step after it is sent. So Scatter moves $p-1$ blocks in the time Bcast moves one: its cost is the lower bound set by the root's link.
Version with SendTo: the root's link, the bottleneck, carries the messages of the $p-1$ calls back to back, r_d pieces of m/r_d elements for the call at distance d: it is busy for $\sum_d (r_d\,L + m\,b)$, and the last message, the block of the neighbour P_{k+1}, arrives when it leaves this link (the farther transfers, started earlier, are over by then):

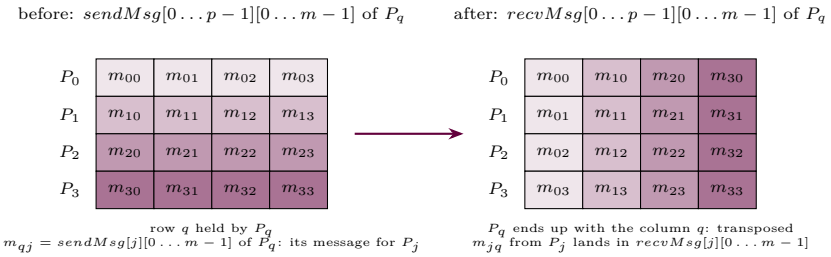
$$T_{\text{SendTo}} = L \sum_{d=1}^{p-1} r_d + (p-1)\,m\,b.$$

With $r_d = 1$, $T_{\text{SendTo}} = (p-1)(L + m\,b)$: the version is the pipelined Scatter itself, message for message. With the $r^\star$ of 1(c), $r_d^\star = \sqrt{(d-1)\,m\,b/L}$: $T_{\text{SendTo}} = \sqrt{L\,m\,b}\,\sum_{d=1}^{p-1}\sqrt{d-1} + (p-1)\,m\,b$, the same transmission term and $\big(\sum_d r_d^\star - (p-1)\big)L$ more latency; a share of order $\sqrt{L/(m\,b)}$ of the total, negligible for large messages, but a pure loss: the pieces fill a pipeline that is already full. So SendTo fills the pipeline as well as Scatter does, the root's link carrying every block exactly once in both, provided the blocks are not cut: pipelining inside a transfer helps a single transfer (Exercises 1 and 2), not a sequence of transfers that already follow each other on the links. For small messages ($m\,b \ll L$), $r^\star = 1$ anyway, and the naive version, $(p-1)\,L + \frac{p\,(p-1)}{2}\,m\,b$, is as good as the pipelined one: with the latency dominating, what counts is the number of messages on the critical path, $p-1$ in both.

Exercise 7

Transposition on a ring

Every processor P_q holds p messages of m elements in a $p \times m$ array $sendMsg[0 \dots p-1][0 \dots m-1]$: the row $sendMsg[j][0 \dots m-1]$ is the message meant for P_j ($sendMsg[q][0 \dots m-1]$ for itself). At the end, every P_q must hold in the row $recvMsg[j][0 \dots m-1]$ of a $p \times m$ array $recvMsg$ the message that P_j sent to it, for every j. Seen as a $p \times p$ array of messages whose row q is held by P_q, the operation transposes the array: P_q ends up with the column q. This exchange is the *all-to-all* of MPI.



a) Design Transposition($sendMsg, recvMsg, m$) on the ring, in two steps. First write Shift(src, dst, m, k): every P_q sends the m elements of its array src to $P_{(q+k) \bmod p}$ and receives in dst those of $P_{(q-k) \bmod p}$, a cyclic shift of the messages by k positions. Then use it: proceed in $p-1$ rounds; in round s, every processor sends the message meant for the processor at distance s.

b) Find its communication cost in terms of p, L, m and b.

c) Count the total number of link crossings that any algorithm must perform, and deduce a lower bound on the cost of a transposition on the ring. Is your algorithm optimal?

Solution

(a)

Shift: the message of every processor travels k hops to the right. At each hop, every processor sends the message it holds and receives the one of its left neighbour (line 3), and the swap of line 4 makes the message just received the one to forward at the next hop: after hop t, P_q holds the message of P_{q-t}; after k hops, the one of P_{q-k}, which it stores in dst (line 5). All the processors do the same thing at the same time, so every link carries exactly one message at every hop: Shift costs $k\,(L + m\,b)$. The copy of line 1 leaves src untouched.

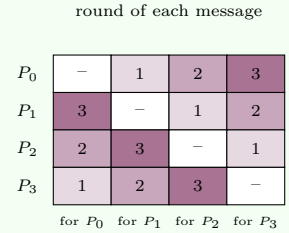
Shift(src, dst, m, k)

```
1: tempS[0 ... m-1] ← src[0 ... m-1]
2: for t = 1, ..., k do
3:     Send(tempS, m) || Recv(tempR, m)
4:     tempS ↔ tempR
5: dst[0 ... m-1] ← tempS[0 ... m-1]
```

Transposition: proceed in $p-1$ rounds. In round s, every processor sends the message meant for the processor s links away, the row $sendMsg[(q+s) \bmod p]$, with a Shift by s positions (lines 4–6); what it receives is the message of P_{q-s}, meant for $P_{q-s+s} = P_q$, which the same call stores in the row $recvMsg[(q-s) \bmod p]$. Line 2 handles the message for itself. A row of a 2-D array is passed by name like any array: $sendMsg[j]$ is the array of the m elements $sendMsg[j][0], \dots, sendMsg[j][m-1]$. The figure shows, for $p = 4$, in which round each message travels.

Transposition($sendMsg, recvMsg, m$)

```
1: q ← MyRank(); p ← NumProcs()
2: recvMsg[q][0 ... m-1] ← sendMsg[q][0 ... m-1]
3: for s = 1, ..., p-1 do
4:     j ← (q + s) mod p          ▶ the message for P_j
5:     i ← (q - s) mod p          ▶ the message from P_i
6:     Shift(sendMsg[j], recvMsg[i], m, s)
```



(b) Round s is a SHIFT by s positions: s hops of one message of m elements on every link, $s\,(L + m\,b)$. In total,

$$T_{comm} = \sum_{s=1}^{p-1} s\,(L + m\,b) = \frac{p\,(p-1)}{2}\,(L + m\,b).$$

(c) The message of P_q for P_{q+s} must cross at least s links, whatever the algorithm; every processor has one message for each distance $s = 1, \ldots, p-1$, so its messages need $\sum_{s=1}^{p-1} s = p\,(p-1)/2$ link crossings, and all the processors together need $p^2(p-1)/2$. The ring has p links and a link carries at most one message per $L + m\,b$: at least $\frac{p^2(p-1)/2}{p} = \frac{p\,(p-1)}{2}$ steps are needed, hence $T_{comm} \geq \frac{p\,(p-1)}{2}\,(L + m\,b)$ for any algorithm that sends the messages whole. The algorithm of (a) reaches this bound: it is optimal, every link is busy at every step.

Exercise 8

Grids, hypercubes and binary trees

Back to the parameters of the lecture, on three other topologies.

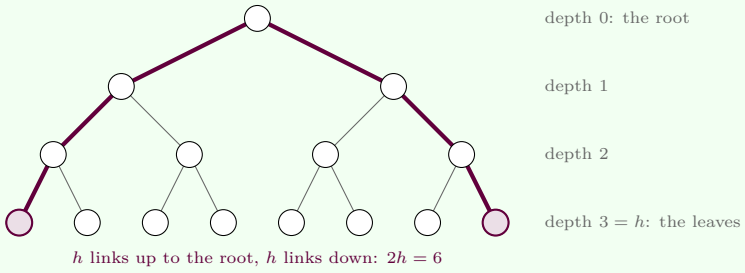
a) What is the diameter of a complete binary tree of height h (the root at depth 0, the leaves at depth h)?

b) On an $n \times n$ grid, how many different shortest paths are there between the processors at positions (x_1, y_1) and (x_2, y_2)?

c) Prove that the bisection width of the d-cube ($p = 2^d$ processors) is 2^{d-1}: exhibit a cut of 2^{d-1} links that splits it into two halves, then show that no cut with fewer links does.

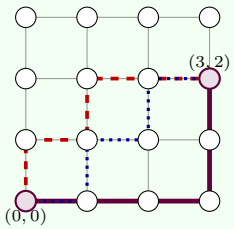
Solution

(a) Two nodes are joined by the path that goes up to their lowest common ancestor and down again; the longest such path joins two leaves whose only common ancestor is the root: h links up, h links down. The diameter is $2h$.



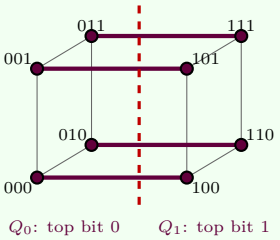
(b) Let $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$. A shortest path never moves away from its destination: it has exactly Δx horizontal steps (all towards x_2) and Δy vertical steps (all towards y_2), $\Delta x + \Delta y$ steps in total, in any order, and every order stays inside the grid since the coordinates stay between those of the two ends. A path is determined by the choice of which of its $\Delta x + \Delta y$ steps are horizontal:

$$\binom{\Delta x + \Delta y}{\Delta x} = \frac{(\Delta x + \Delta y)!}{\Delta x!\;\Delta y!} \quad \text{shortest paths.}$$



$\Delta x = 3$, $\Delta y = 2$: 5 steps, 2 of them vertical
$\binom{5}{2} = 10$ shortest paths, three of them drawn:
RRRUU, URURR, RURUR

(c) *A cut of 2^{d-1} links.* Split the nodes by their top bit: Q_0 holds the 2^{d-1} nodes $0b_{d-2}\ldots b_0$ and Q_1 the 2^{d-1} nodes $1b_{d-2}\ldots b_0$; each is a $(d-1)$-cube, and the only links between them are the 2^{d-1} links of the top dimension, between each node of Q_0 and its twin in Q_1. Removing them splits the d-cube into two halves: the bisection width is at most 2^{d-1}.



the $2^{d-1} = 4$ links of the top dimension (thick) separate the two $(d-1)$-cubes Q_0 and Q_1

No cut does better. We prove by induction on d the following lemma: in a d-cube, every set S of $s \leq 2^{d-1}$ nodes has at least s links *leaving* it (links with exactly one end in S). A half of a bisection is a set of 2^{d-1} nodes, and the links leaving it are the links of the cut: the lemma gives at least 2^{d-1} of them.
Proof of the lemma. For $d = 1$, S is one of the two nodes and the single link leaves it. For $d > 1$, split the cube into Q_0 and Q_1 as above and let $S_0 = S \cap Q_0$ and $S_1 = S \cap Q_1$, with $s_0 + s_1 = s$ nodes. The links leaving S are of three kinds: those leaving S_0 inside Q_0, those leaving S_1 inside Q_1, and the links of the top dimension whose two ends, a node of Q_0 and its twin in Q_1, are not both in S or both outside: there are at least $|s_0 - s_1|$ of these, since if every node of S_0 had its twin in S_1 we would have $s_1 \geq s_0$, and symmetrically.

- If $s_0 \leq 2^{d-2}$ and $s_1 \leq 2^{d-2}$, the induction hypothesis in the two $(d-1)$-cubes gives at least $s_0+s_1+|s_0-s_1| \geq s$ links.
- Otherwise one of the two, say s_0, exceeds 2^{d-2}, and then $s_1 = s - s_0 < 2^{d-2}$. The links leaving S_0 inside Q_0 are also the links leaving its complement $Q_0 \setminus S_0$, which has $2^{d-1} - s_0 < 2^{d-2}$ nodes: by induction there are at least $2^{d-1} - s_0$ of them. With at least s_1 links leaving S_1 and at least $s_0 - s_1$ links of the top dimension, we get at least $2^{d-1} - s_0 + s_1 + s_0 - s_1 = 2^{d-1} \geq s$ links.

Hence the bisection width of the d-cube is exactly $2^{d-1} = p/2$, as stated in the table of the lecture; the cut of one dimension is optimal.